

HOTEL BOOKING SYSTEM

Mini Project using Python

Programming Concepts (CCS1300)

K.A. Sahiru Sandeepa (12B)

CIT-25-02-0289

Sri Lanka Technology Campus

07th of March 2026

Abstract

The Sunset View hotel booking system is a strong desktop application developed with Python 3 and the Tkinter Graphical User Interface (GUI) toolkit. The project stored the number of rooms reserved by a hotel receptionist and eliminates the need for inefficient paper notes that are prone to human error. Comprehensively validating all users' input while allowing them to add, view and delete guests' bookings. It uses core python programming concepts such as control flow like if, else and logic loops and a mix of specialized data structures like list, dicts, tuples and sets to meet modern software, application requirements.

Moreover, using the file-handling capabilities of Python, it ensures data persistence by invisibly reading from and writing to a Comma Separated Values (CSV) file. The application also has a proper ttk-mock styled user interface button and error handling combine at the right spot.

Introduction

The system can be used to manage hotel rooms: the allocation of reservations is essential for the hospitality sector, as it plays an important role in revenue and customer satisfaction. Simple paper-based or spreadsheet systems generate human error and details could be lost meaning a restaurant could double book tables, for example.

This mini project introduces the desktop application "Sunset View Hotel Booking System," designed to address these problems. The project was taken as a part of the Programming Concepts module allowing me to further my understanding of fundamental programming topics in Python, including basic GUI design and layout customization, and various data structures, alongside file input/output manipulation. The software effectively executes the administrative processes, enabling hotel staff to use intelligent interfaces. It avoids double bookings by employing intelligent data validation and keeps a continuous reference for every guest that can be easily referenced.

System Overview

The system is designed to be an independent, local based class end up on a desktop application that operates and runs multiple single window clients. The system, upon launch, immediately checks for any existing local data files and loads whatever reservations were made into the memory as well as its visual data grid.

This is the architecture of the project:

- **User Input** - The user interacts with the Tkinter graphical interface. GUI input guests' details (Name, Phone, Room, Days, Amount, Payment Method) via text entries and dropdown menus.
- **Validation Engine** - The system evaluates the input using control structures to ensure no fields are empty, numerical fields contain valid digits, and the requested room is not already booked.
- **Memory Management** - Validated data is structured into dictionaries and appended to a list. The room number is added to a set to ensure for future availability checks.
- **Storage/File Handling** - The in-memory data structures are mapped and written to a .csv file (hotel_bookings.csv) to ensure the data is continuously stored on the hard drive.
- **GUI Update** - The Treeview component refreshes to display the newly added or deleted records to the user.

Tools and Technologies Used

To build this project, I used a set of technologies that are straightforward to use and fit the needs of a desktop application perfectly.

This project is built using Python 3.x for its core logic, depending on the Tkinter framework and its ttk extension to design the interactive graphical user interface. It utilizes built in Python modules like CSV and OS for data storage and file management, all of which were developed and tested using a standard Integrated Development Environment (IDE).

Programming Language - Python 3.x

Python 3.x is a newer iteration of Python designed for general purpose programming. Python is a high-level programming type that is widely used in many fields including web development, data science, automation and artificial intelligence due to its intuitive syntax and easy readability.

GUI Framework

- **Tkinter**

Tkinter is the built-in graphical user interface (GUI) package for Python. It enables developers to build desktop applications with windows, buttons, labels, text fields and other graphical parts.

- **Ttk (Tkinter themed widgets)**

This module comes as an extension to the tkinter, which provides themed widgets, they have a more modern and native look. It includes better components such as buttons, comboBoxes and progress bars with improved styling.

Standard Libraries

- **CSV (Comma-Separated Values)**

CSV is a straightforward document format to store table data as plain text. Values are separated by commas, making it easy to read/write data and exchange data between programs like spreadsheets and databases.

- **OS (Python standard library)**

The **OS** module provides functions to interact with the operating system. It can be used to check whether files or directories exist before loading data.

- **MessageBox**

The messagebox module is part of Tkinter and is used to display dialog boxes in GUI applications. It shows messages such as success, warnings, and errors to inform the user.

Development Environment (IDE)

- **Visual Studio Code (VS code)**

Visual Studio Code is a free, open-source code editor developed by Microsoft. It supports a wide range of programming languages and frameworks, offering features like debugging, syntax highlighting, and integrated version control. It is lightweight and highly expandable among the most popular development tools worldwide.

Implementation Details

Project Description and Features

The Sunset View Hotel Booking System is built to handle the daily operational needs of a hotel front desk. These are the key features,

- **Add Booking** - Users can register a new guest by filling out a form.
- **Delete Booking** - Users can select an existing record from the grid and permanently delete it from both the application and the database.
- **Input Validation** - Prevents the submission of empty forms, restricts day/phone/room inputs to numeric values dynamically, and prevents double-booking of the same room.
- **Permanent Storage** - Data is saved to a CSV file so that it remains available even after the application is closed and reopened.
- **Clear Interface** - A dedicated button to clear input fields for rapid sequential data entry.

Used GUI components

- **Main Window (tk.Tk)**

The primary window that holds the entire application.

- **Frames (tk.Frame)**

Used to organize the layout into left (input form) and right (data table) sections.

- **Labels (tk.Label)**

Used for the title and the text prompts ("Guest Name: ")

- **Entry Fields (tk.Entry)**

The text boxes where users type in guest details.

- **Combobox (ttk.Combobox)**

The dropdown menu used to select the "Payment Method".

- **Buttons (tk.Button)**

The clickable buttons are used to submit, clear, or delete data.

- **Treeview (ttk.Treeview)**

The table on the right side displays all the saved bookings in columns.

- **Message Boxes (messagebox)**

The pop-up windows used to show success, warning, and error alerts to the user.

Explanation of How Data Structures Are Used

The system leverages four distinct Python data structures to optimize performance and logic.

- **Lists (`bookings_list = []`)**

Used for the storage of multiple guest records, each stored in a specific and ordered sequence. This is the main memory database that gets iterated through when saving the records to the CSV file.

- **Dictionaries (`booking_dict = {...}`)**

Used for the storage of structured and labeled records for each individual booking, such as grouping "Name," "Phone," and "Room" together. This code is extremely readable and works perfectly with the `csv.DictWriter` and `csv.DictReader` methods.

- **Sets (`booked_rooms_set = set()`)**

Used specifically for tracking the rooms that are already occupied. Since sets prevent duplicate entries and are very efficient at looking up entries, it is the best choice for determining if a room has already been booked.

- **Tuples (`COLUMNS = (...)`)**

Used to store the column headers for the Treeview and CSV file. Because headers are constant and should never be altered during runtime, a tuple is the safest and most memory efficient structure to use.

Explanation of File Handling

File handling is implemented using Python's built in csv module to read and write tabular data.

- **Reading (load_data())**

The system uses `os.path.exists()` to verify if `hotel_bookings.csv` exists. If it does, it opens the file in read mode ('r') and uses `csv.DictReader` to parse the rows back into the application's Lists and Sets, and dynamically populates the Treeview.

- **Writing (save_data())**

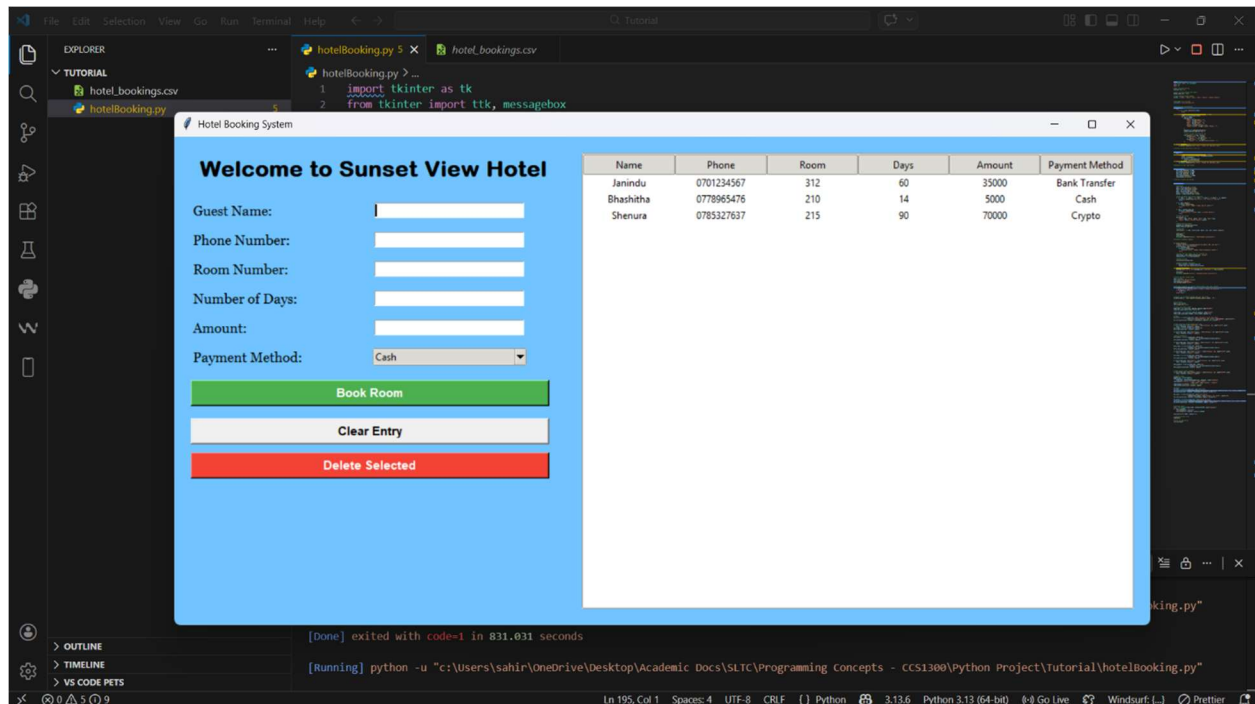
Triggered after every 'Add' or 'Delete' operation. The system opens the file in write mode ('w'), which overwrites the previous state. It uses `csv.DictWriter` to carefully format the dictionary items back into CSV rows.

- **Error Handling**

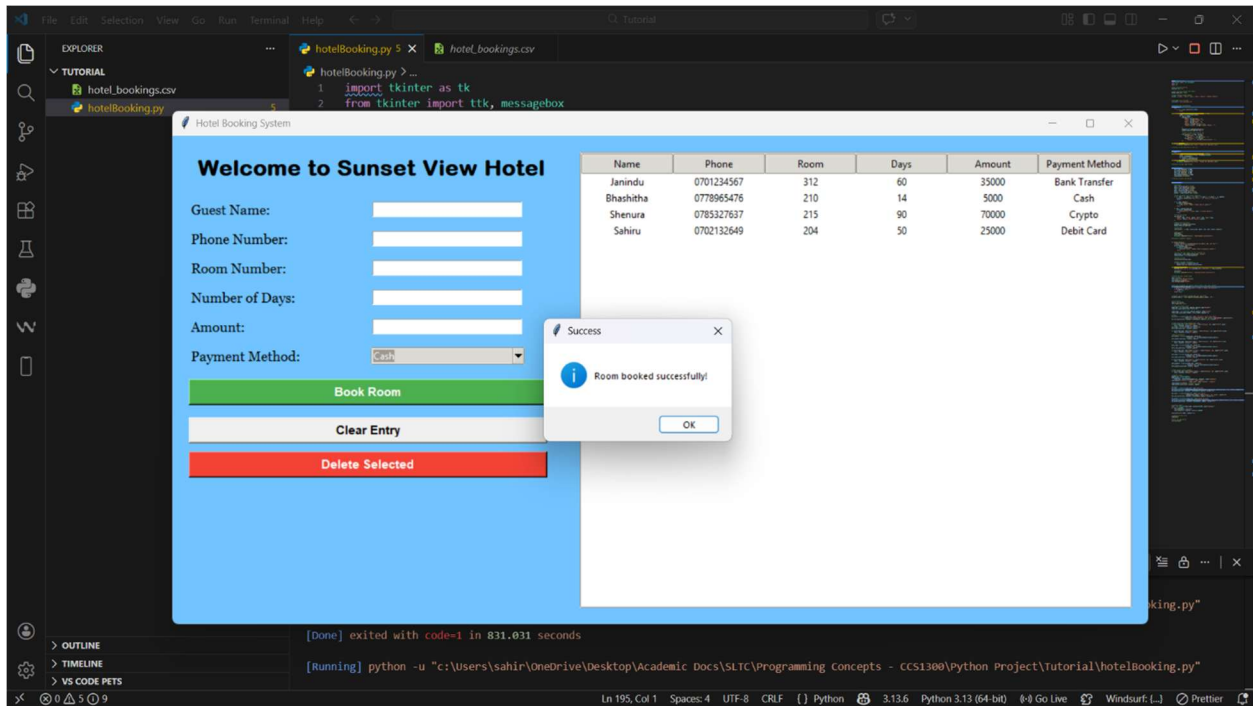
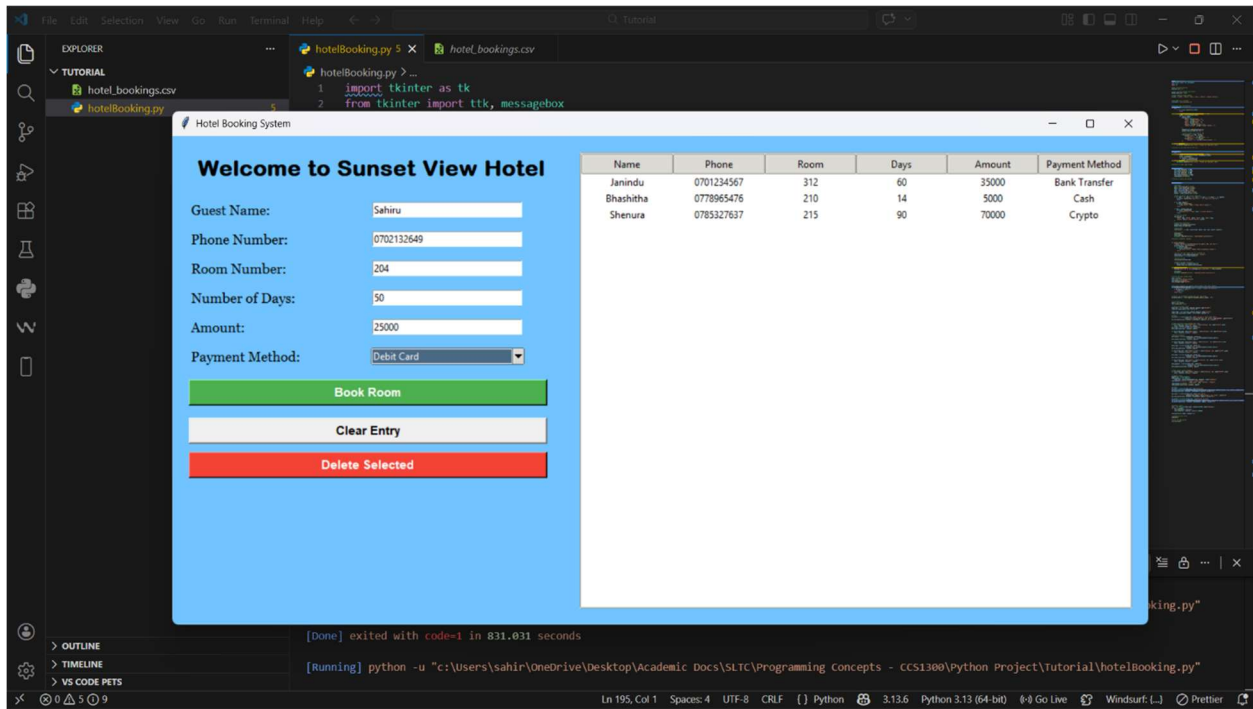
File operations are wrapped in try-except blocks. If an unexpected OS error occurred the system catches the exception and displays an error messagebox instead of crashing.

Screenshots of GUI

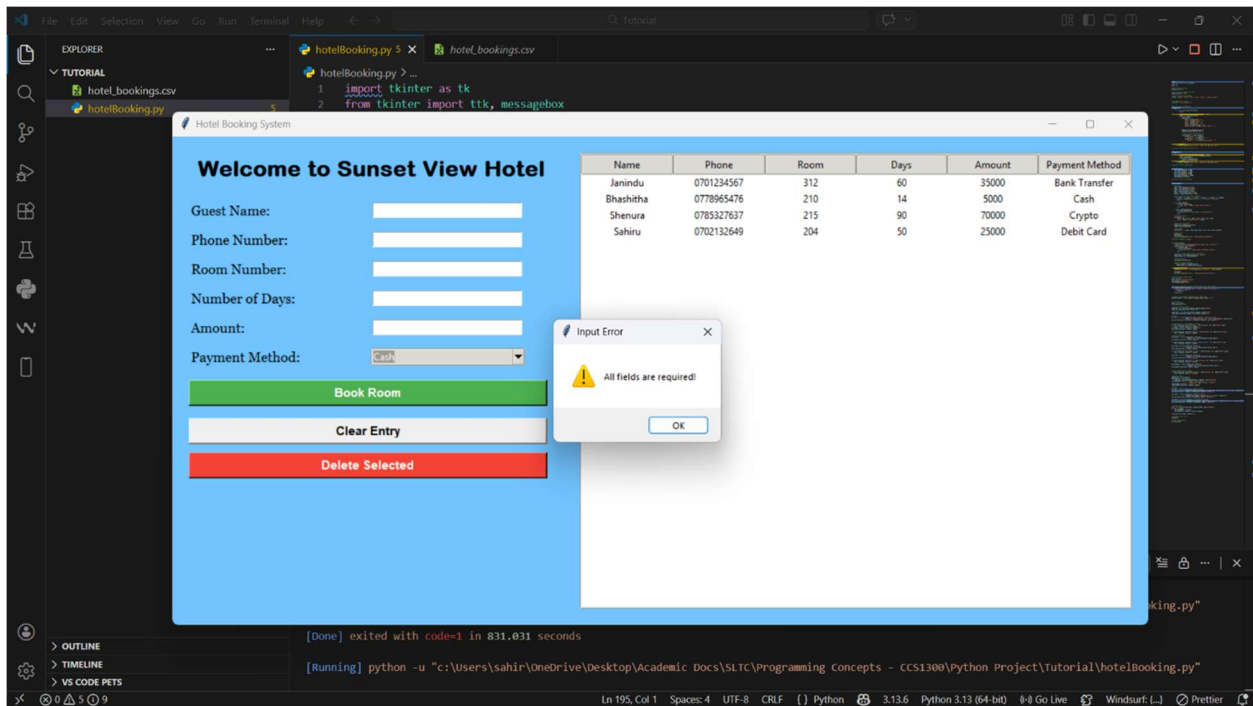
Main Window Design



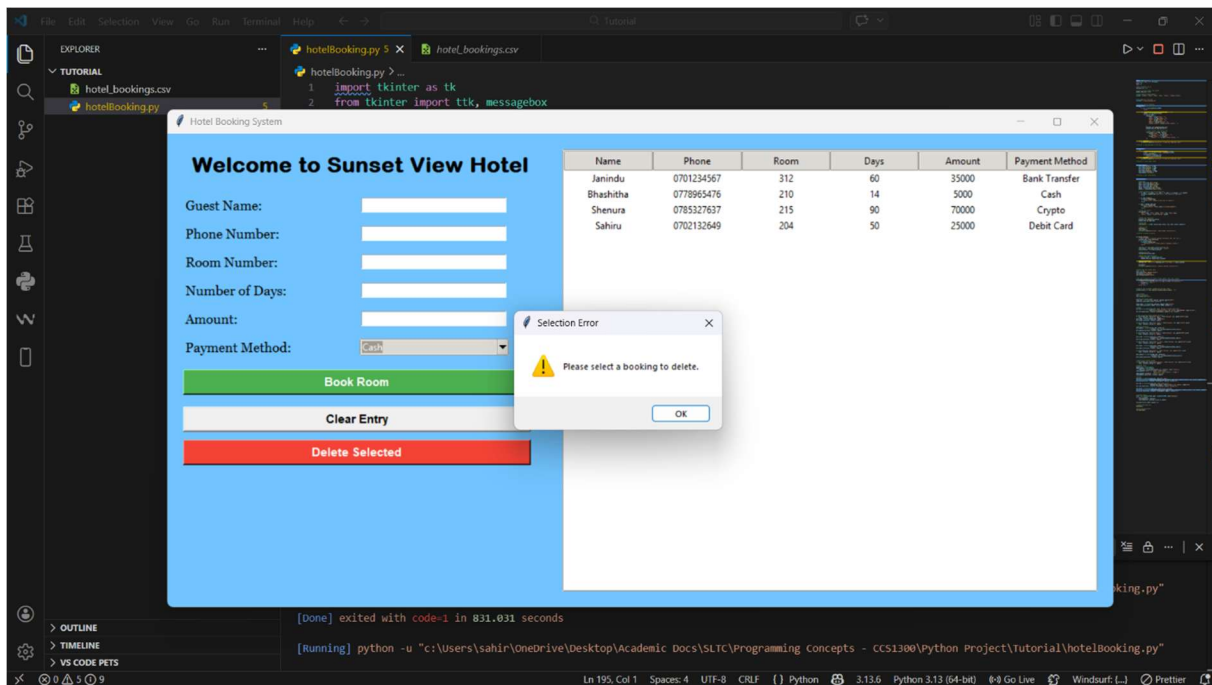
Adding a new guest's details



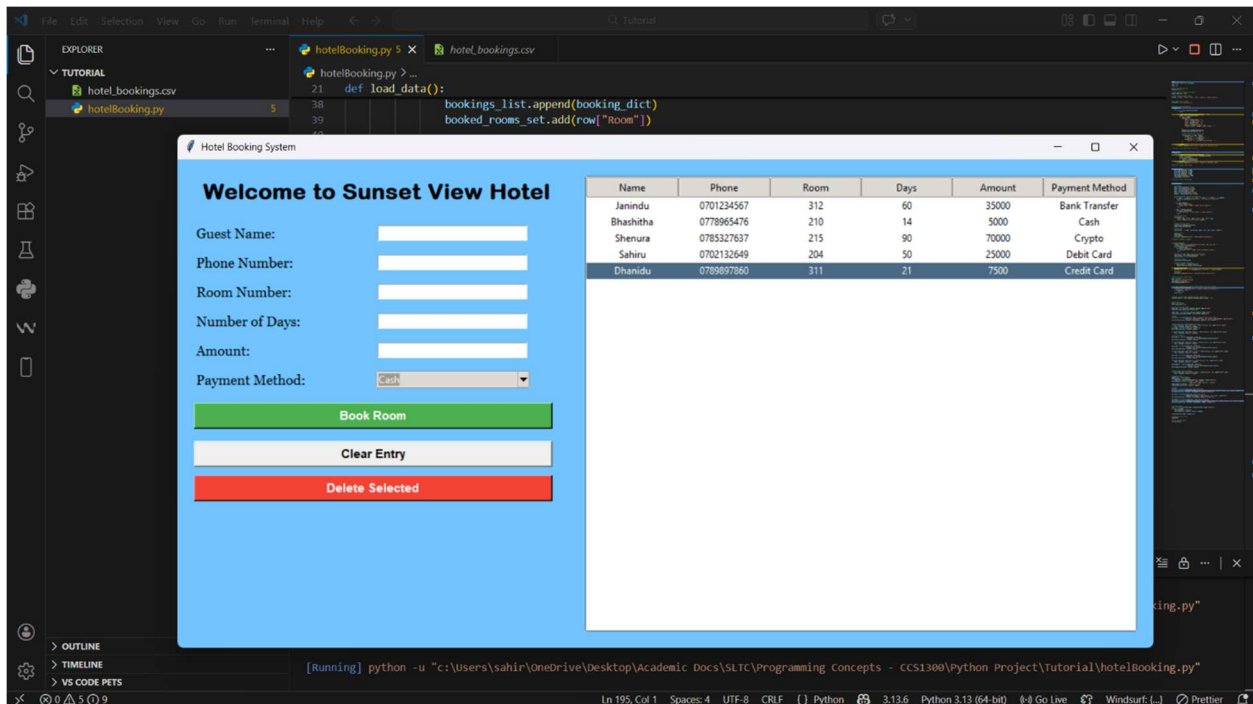
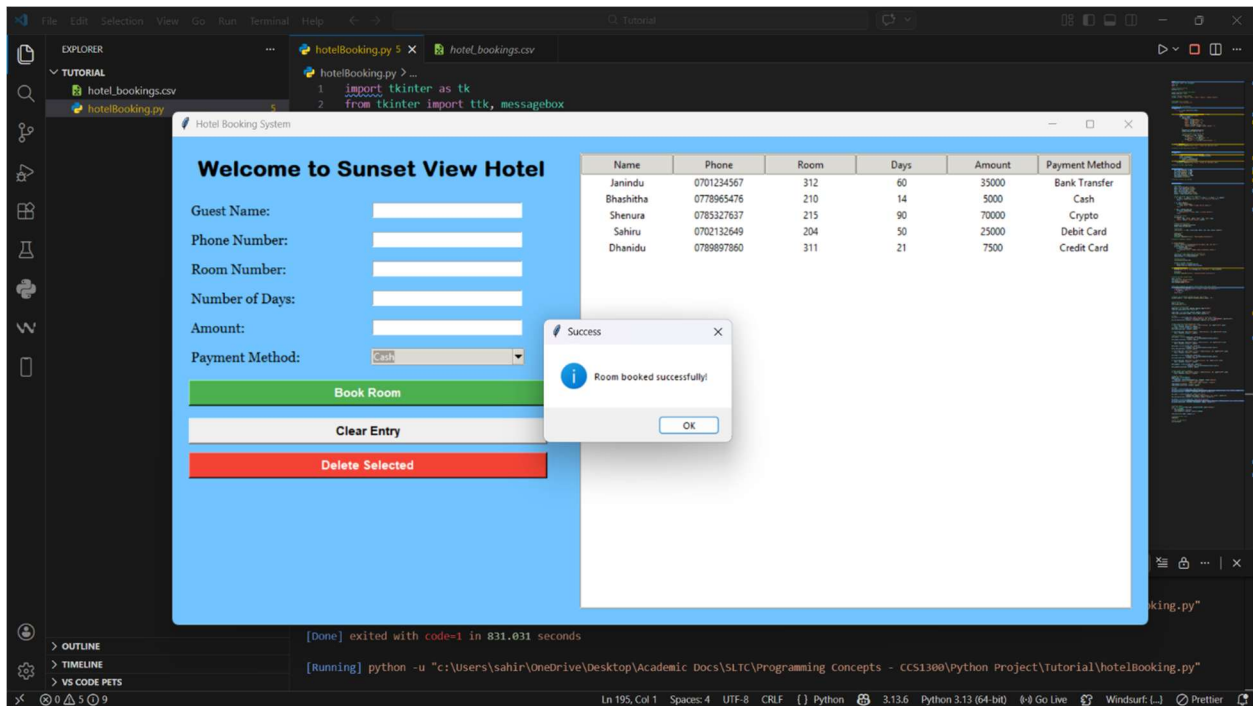
When booking without completing the fields

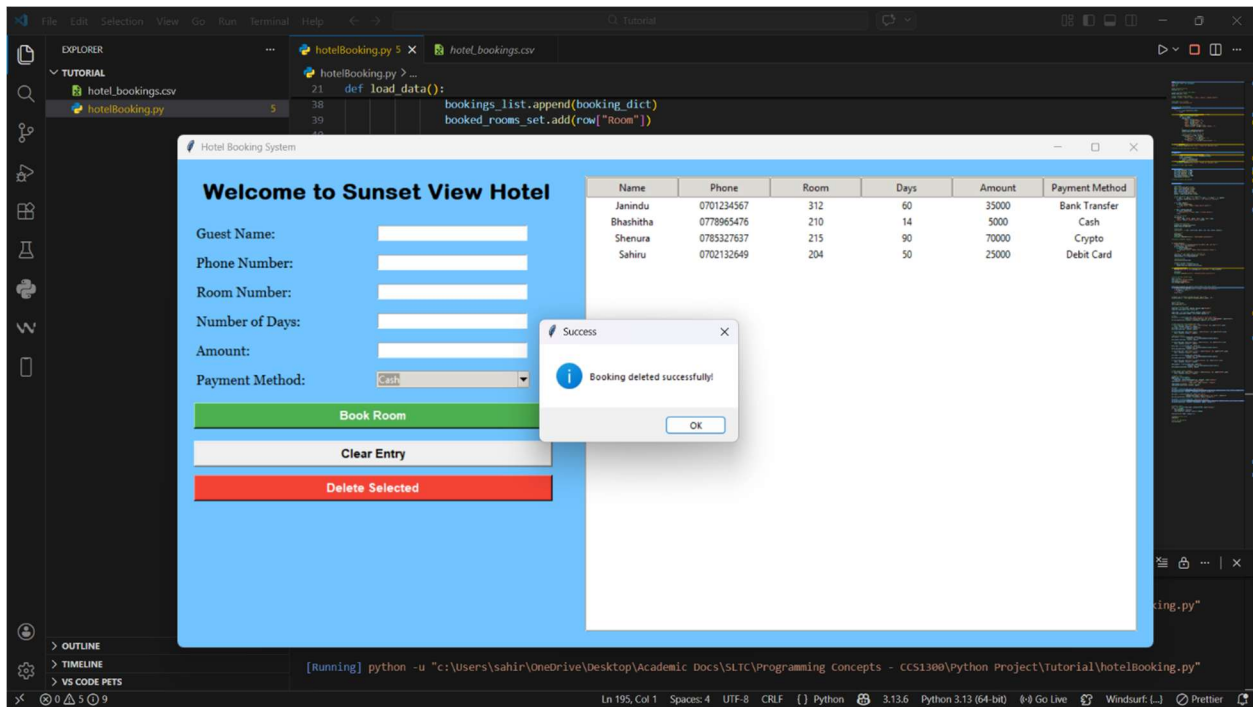


When delete without selecting a booking

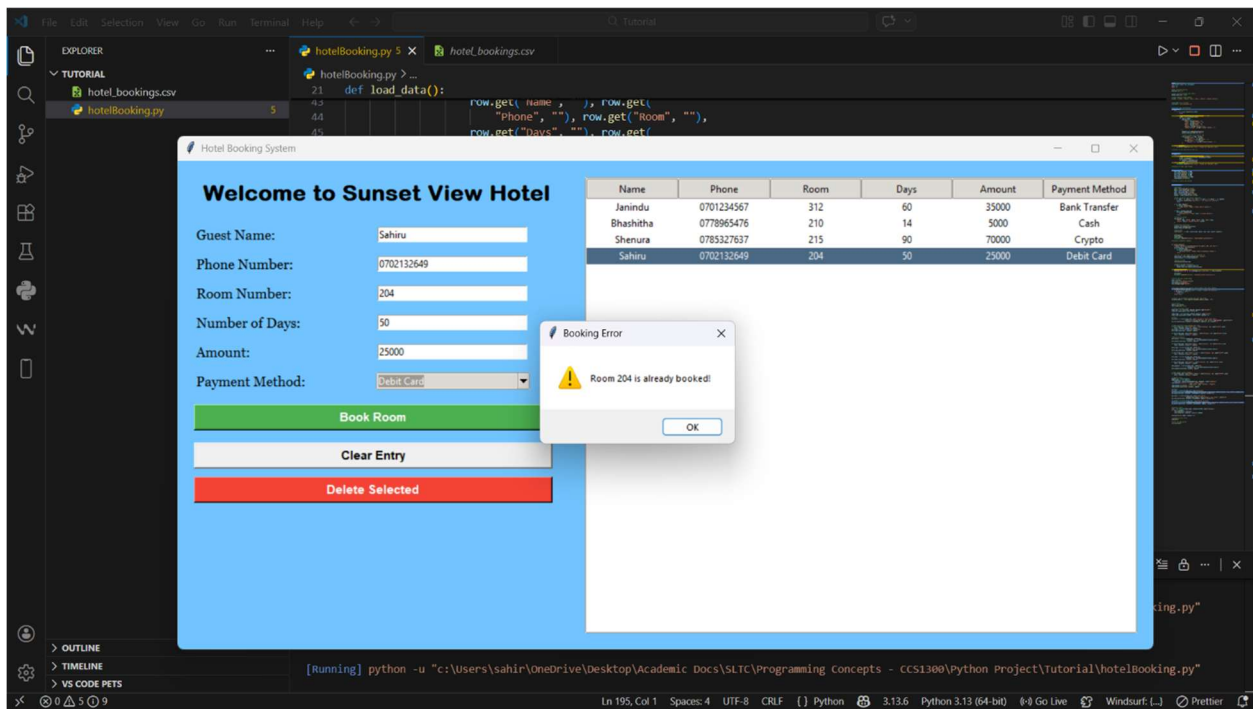


Adding another booking and delete selected booking





Attempt to book a room number that is already booked in the grid.



Use of Python Concepts

Variables - Extensively used for temporary storage of user input values (name, phone, room), system state (payment_var), and constants (FILE_NAME).

Functions - The project is highly modular, using more than the minimum required 5 functions.

- load_data() - Initializes the app with historical data.
- save_data() - Handles file output.
- clear_inputs() - Resets the GUI state.
- add_booking() - Core business logic for creating a record.
- delete_booking() - Logic for safely removing a record.
- validate_numeric_input(P) - A specialized callback function that dynamically filters keystrokes in the Tkinter UI to ensure only numbers are typed into specific fields.

Testing

To ensure the system is reliable, the following manual test cases were executed during development.

Test Case	Expected Result	Actual Result	Status
Fill all fields with valid data and click "Book Room".	Booking added to grid, inputs clear, success message shown.	As expected	Pass
Leave 'Guest Name' blank and click "Book Room".	Warning messagebox appears ("All fields are required!").	As expected	Pass
Book a room number that is already in the grid.	Warning messagebox appears ("Room X is already booked!").	As expected	Pass
Type alphabetical characters into the 'Phone Number' field.	Keystrokes are rejected by the UI validation function.	As expected	Pass
Close the application completely and reopen it, select an item in the Treeview and click "Delete".	Previously added bookings loaded and item is removed from the grid.	As expected	Pass

Future Improvements

Although the present system meets all the initial requirements, there are certain modifications that could be made to the system in the future.

- **Database Integration**

Upgrading the file handling to a database like SQLite3, which would make the system more scalable.

- **Search and Filter**

Adding a search bar to filter the guests based on their 'Name' and 'Room Number', which would be very useful as the number of guests increases.

- **Update Functionality**

Allowing the user to double-click the record to make changes to the details, like the number of days, without having to delete the record and re-enter the data.

- **Receipt Generation**

Automatically generating a receipt for the guest after the booking has been successful.

Conclusion

Sunset View Hotel Booking System is a working example of the core programming principles introduced in Programming Concepts module. With Python native data structures, modular function design and precise flow control, this application provides reliable backend logic. Within the context of using Tkinter demonstrated via a user-friendly and validated interface, introduced file-handling mechanisms that would persist data from the GUI documents are saved to local files for future retrieval cloning a working software solution in real use.